

GUÍA DEL ALUMNO · JAVASCRIPT EN EL NAVEGADOR

Vuestra calculadora, paso a paso

Buenas, soy Ramón Abramo, profesor con más de 25 años de experiencia docente. Esta guía os lleva de la mano para programar la calculadora de tres formas distintas. Un paso por apartado, una prueba al final de cada paso y las soluciones en un anexo al final, por si os atascáis.

Solución 0 · 9 pasos | Solución 1 · 7 pasos | Solución 2 · 8 pasos

Ramón Abramo · Guía guiada para el alumnado



Lo que necesitáis tener abierto

1. Abrid la carpeta de la solución que vais a hacer en vuestro editor (Visual Studio Code, por ejemplo).
2. Dentro tenéis `index.html`, `main.css` y `app.js`. **Solo vais a tocar** `app.js`, que es la plantilla con los comentarios `TODO`.
3. Abrid `index.html` en el navegador (doble clic o con la extensión Live Server).
4. Pulsad `F12` y abrid la pestaña **Consola**. Ahí aparecen los errores y ahí podréis probar cosas[1].



Ojo: el HTML y el CSS ya están hechos. Si cambiáis los atributos `data-` de los botones, el JavaScript dejará de encontrarlos.

El ciclo que vais a repetir mil veces

Escribir → **Guardar** → **Recargar (F5)** → **Probar**

Parece una tontería, pero la mitad de las dudas que me llegan son de alguien que no ha guardado o no ha recargado. Si cambiáis el código y la página sigue igual, sospechad primero de eso.

Y una regla de oro: **no paséis al siguiente paso hasta que la prueba del actual salga bien**. Si acumuláis cinco pasos sin probar y algo falla, no sabréis cuál de los cinco ha sido.



Así está organizado cada paso



Objetivo: lo que tiene que pasar en pantalla cuando terminéis el paso.

Dónde: en qué bloque `TODO` de vuestro `app.js` tenéis que escribir.

Pistas, en orden

Una lista de lo que hay que escribir, contado con palabras. Intentad convertirla en código vosotros solos.



Comprobadlo: qué teclas pulsar y qué tiene que salir.



Si no funciona: los fallos más típicos de ese paso y cómo arreglarlos.



¿Atascados? Cada paso os dice en qué apartado del anexo final está su solución.

He puesto las soluciones al final a propósito. Copiarlo sin pensar es como mirar la última página de un sudoku: lo terminas, pero no has aprendido nada.

☐ **¡Hecho! Funciona en pantalla**

Cuando la prueba os salga bien, marcad la casilla «Hecho» con el boli. Así sabréis siempre por dónde ibais.

ANTES DE EMPEZAR · EL HTML QUE YA TENÉIS



Cada botón lleva una etiqueta pegada

```
<output id="display" class="display">0</output>

<div class="teclado">
  <button data-accion="borrar">C</button>
  <button data-operador="/">/</button>
  <button data-numero="7">7</button>
  ...
  <button data-operador="+">+</button>
  <button data-numero="0">0</button>
  <button data-accion="igual">=</button>
</div>
```

Hay tres tipos de botones, y se distinguen por su atributo `data-`:

- `data-numero` → del `0` al `9` (10 botones).
- `data-operador` → `+` `-` `*` `/` (4 botones).
- `data-accion` → `igual` y `borrar` (2 botones).



Para entenderlo: son como las etiquetas de los botes de la cocina: todos los botes son iguales, pero la etiqueta os dice qué hay dentro.

Desde JavaScript, `data-numero="7"` se lee como `boton.dataset.numero` y vale `'7'`, **con comillas**: es texto[2].

ANTES DE EMPEZAR · LA MEMORIA DE LA CALCULADORA



Cuatro cosas que la calculadora tiene que recordar

Pensad en lo que pasa cuando tecleáis `15` `+` `4` `=`. Para poder decir `19`, la calculadora necesita tener apuntado:

1. **Lo que hay ahora en pantalla** → `4`
2. **El número de antes** → `15`
3. **La operación pendiente** → `+`
4. **Una bandera** que diga: «el próximo dígito empieza un número nuevo, no lo pegues al de antes».



Para entenderlo: un camarero con su libreta. Apunta el primer plato, qué queréis hacer con él y luego el segundo. Si no apunta, se le olvida.

QUÉ GUARDA	SOLUCIÓN 0	SOLUCIONES 1 Y 2
Lo de la pantalla	la propia pantalla	`operandoActual`
El número de antes	`numeroAnterior`	`operandoAnterior`
La operación	`operadorPendiente`	`operacion`
La bandera	`empezarNumero`	`reiniciarDisplay`

Estas variables **ya están declaradas** en vuestra plantilla. No las borréis ni las cambiéis de nombre.

Sin la bandera, al teclear `15 + 4` veríais `154`. Y eso no es una suma.



Las piezas de JavaScript que vais a usar

Buscar botones

`querySelectorAll` busca **todos** los elementos que cumplen un selector[3]. `querySelector` devuelve solo **el primero**[4].

```
document.querySelectorAll('[data-numero]');
document.querySelector('.teclado');
```

Recorrer una lista

`forEach` hace lo mismo con cada elemento de la lista, uno tras otro[5].

```
numeros.forEach(boton => {
  // "boton" es cada uno
});
```

Reaccionar al clic

`addEventListener` dice: «cuando te hagan clic, ejecuta esto»[6].

```
boton.addEventListener('click', () => {
  // esto pasa al hacer clic
});
```

Y para escribir en la pantalla de la calculadora, `pantalla.textContent = '7'`. Para leerla, `pantalla.textContent` a secas[7]. Siempre es texto.

ELIGE TU CAMINO



Tres soluciones, la misma calculadora

Elegid la que vayáis a hacer. Mi consejo: empezad por la 0, seguid con la 1 y dejad la 2 para el final. Cada una se apoya en lo que aprendéis en la anterior.

0 · La solución de clase

Tres grupos de botones y una función `calcular()` que recibe los datos y devuelve el resultado.

1 · Un escuchador por botón

Cada botón avisa a una función con nombre propio: `agregarNumero`, `calcular`…

2 · Delegación de eventos

Un solo escuchador en todo el teclado. Las funciones son las mismas que en la 1.

Al final de la guía tenéis la lista de comprobación antes de entregar, la traducción de los mensajes de error de la consola y un anexo con las soluciones de cada paso.



0

SOLUCIÓN 0 · CARPETA VERSION-0-SOLUCION-CLASE

La solución de clase

Primero construiremos una función que solo hace cuentas. Después, agruparemos los botones en tres montones (números, operadores y acciones) y le daremos a cada montón su escuchador.

1 · calcular() 2 · Botones 3 · Hito 1 4 · Hito 2 5 · Hito 3 6 · Hito 4 7 · Hito 5 8 · Hito 6 9 · Hito 7

SOLUCIÓN 0 · PASO 1 DE 9 · PREPARATIVOS



La función `calcular()`



Objetivo: una función que recibe dos números y una operación, y devuelve el resultado. No toca la pantalla ni ninguna variable de fuera.

Dónde: bloque `TODO 0.1`, dentro de `function calcular(numero1, numero2, operacion)`.

Pistas, en orden

1. Escribid un `switch (operacion)`[8].
2. Un `case` para cada operador: `'+'`, `'-'`, `'*'` y `'/'`. En cada uno, guardad la cuenta en `resultado`.
3. Terminad cada `case` con `break`.
4. En la división: si `numero2 === 0`, `resultado = 'Error'`; si no, dividid[9].
5. El `return resultado;` ya está escrito al final. No lo toquéis.



Para entenderlo: una exprimidora. Le dais dos naranjas y le decís «zumo». Os da zumo. No le importa de dónde vienen las naranjas.



Comprobadlo: esta función se puede probar sin tocar ni un botón. Recargad y escribid en la consola:

`calcular(7, 8, '+')` → `15`
`calcular(9, 4, '-')` → `5`
`calcular(8, 0, '/')` → `'Error'`



Si no funciona: `calcular(9, 4, '+')` da `5` en vez de `13`: os falta un `break` y el código se «cae» al caso de abajo. Si da `0`, revisad que escribís `'+'` con comillas en el `case`.



¿Atascados? La solución de este paso está en el anexo, apartado **S0-1**. Intentadlo antes de mirar.

☐ **¡Hecho! Funciona en la consola**



Comprobad que encontráis los botones



Objetivo: tener los botones agrupados en tres listas y la pantalla localizada.

Dónde: bloque `TODO 0.2`.

Este paso ya viene escrito en la plantilla. Aun así, no os lo saltéis: quiero que entendáis cada línea.

```
const numeros    = document.querySelectorAll('[data-numero]');
const operadores = document.querySelectorAll('[data-operador]');
const acciones   = document.querySelectorAll('[data-accion]');
const pantalla   = document.querySelector('output');
```

Los corchetes `[data-numero]` significan «cualquier elemento que tenga ese atributo, valga lo que valga».



Comprobadlo: en la consola escribid:

`numeros.length` → `10`
`operadores.length` → `4`
`acciones.length` → `2`
`pantalla` → os marca la pantalla de la calculadora.



Si no funciona: si alguno da `0`, revisad que el selector está bien escrito (`data-numero`, no `data-numeros`) y que lleva corchetes.

- [ ] **¡Hecho! Salen 10, 4 y 2**

SOLUCIÓN 0 · PASO 3 DE 9 · HITO 1



Escribir números en pantalla



Objetivo: al pulsar los números, aparecen en pantalla uno detrás de otro. Sin ceros delante: nada de `07`.

Dónde: bloque `TODO 1`, dentro de `numeros.forEach( ... )`.

Pistas, en orden

1. Leed el dígito pulsado: `const valor = numero.dataset.numero;`
2. Preguntad: ¿la pantalla muestra `'0'` **o** `empezarNumero` es `true`? El «o» se escribe `||` [10].
3. Si es así: **sustituid** la pantalla por `valor` y bajad la bandera (`empezarNumero = false`).
4. Si no: **pegad** el dígito detrás con `+=`.

Como `textContent` es texto, `'1' + '2'` da `'12'`. Justo lo que queremos.



Comprobadlo: pulsad `1` `2` `3` → se ve `123`.
Recargad y pulsad `0` `0` `7` → se ve `7`, no `007`.



Si no funciona: si no pasa nada al pulsar, mirad la consola: seguramente hay un error en rojo. Si se ve `07`, la comparación con `'0'` no está bien (¿le habéis puesto comillas?).



¿Atascados? La solución de este paso está en el anexo, apartado **S0-3**. Intentadlo antes de mirar.

☐ **¡Hecho! Escribo 123**

SOLUCIÓN 0 · PASO 4 DE 9 · HITO 2



Guardar el primer número al pulsar un operador



Objetivo: al pulsar `+`, la calculadora apunta el número de la pantalla y la operación. El siguiente dígito empieza un número nuevo.

Dónde: bloque `TODO 2`, dentro de `operadores.forEach( ... )`.

Vamos a hacer primero una **versión sencilla**. En el hito 5 la ampliaremos.

Pistas, en orden

1. Leed el operador: `const valor = operador.dataset.operador;`
2. Guardad lo de la pantalla en `numeroAnterior`, pero **convertido a número** con `Number( ... )` [11].
3. Guardad `valor` en `operadorPendiente`.
4. Levantad la bandera: `empezarNumero = true`.



Comprobadlo: escribid `15`, pulsad `+`, escribid `4` → se ve `4`, no `154`.
En la consola: `numeroAnterior` → `15` y `operadorPendiente` → `'+'`.



Si no funciona: si se ve `154`, no estáis levantando la bandera, o la estáis poniendo a `false` en lugar de `true`.



¿Atascados? La solución de este paso está en el anexo, apartado **S0-4**. Intentadlo antes de mirar.

☐ **¡Hecho! Veo el 4 y la consola dice 15**

SOLUCIÓN 0 · PASO 5 DE 9 · HITO 3



Resolver la cuenta con el igual



Objetivo: al pulsar `=`, se hace la cuenta pendiente y se ve el resultado.

Dónde: bloque `TODO 3`, dentro de `acciones.forEach( ... )`.

Pistas, en orden

1. Leed la acción: `const valor = accion.dataset.accion;`
2. Escribid un `switch (valor)` con un `case 'igual':`. El de borrar lo haremos en el hito 6.
3. Dentro: si `operadorPendiente` `=/=` `null`...
   - convertid la pantalla a número y guardadla en `numeroActual`,
   - llamad a `calcular(numeroAnterior, numeroActual, operadorPendiente)` y guardad lo que devuelve en `numeroAnterior`,
   - pintad `numeroAnterior` en la pantalla.
4. Fuera del `if`: `operadorPendiente = null` y `empezarNumero = true`. Y el `break`.



Comprobadlo: `7` `+` `8` `=` → `15`.
Pulsad `=` sin haber pulsado operador: no debe pasar nada raro.



Si no funciona: si sale `78`, estáis sumando textos: os falta el `Number( ... )` en algún sitio. Si sale `0`, revisad que el paso 1 funciona en la consola.



¿Atascados? La solución de este paso está en el anexo, apartado **S0-5**. Intentadlo antes de mirar.

☐ **¡Hecho! 7 + 8 = 15**



Resta, multiplicación y división



Objetivo: que las cuatro operaciones funcionen desde los botones.

Dónde: no hay que escribir nada nuevo.

¿Por qué? Porque en el paso 1 ya pusisteis los cuatro `case` dentro de `calcular()`. Esa es la gracia de tener la cuenta en una función aparte: los botones no saben ni les importa qué operación es. Le pasan el operador a `calcular()` y ella se apaña.

Este paso es solo de comprobación. Así de simple.



Comprobadlo: `9 - 4 =` → `5`
`6 * 3 =` → `18`
`20 / 4 =` → `5`



Si no funciona: si una de las tres falla y las otras no, el problema está en ese `case` de `calcular()`. Volved al paso 1 y probadla en la consola.

☐ **¡Hecho! Salen 5, 18 y 5**

SOLUCIÓN 0 · PASO 7 DE 9 · HITO 5



Operaciones encadenadas



Objetivo: si tecleáis `5 + 3 +`, al pulsar el segundo `+` la pantalla ya muestra el subtotal `8`.

Dónde: volvemos al bloque `TODO 2` (operadores) y lo ampliamos.

Pistas, en orden

1. Antes de guardar nada, preguntad: ¿había ya una operación pendiente (`operadorPendiente !== null`) **y** el usuario ha escrito un número nuevo (`empezarNumero !== true`)?
2. **Sí** → haced lo mismo que en el igual: `numeroActual`, `calcular( ... )` y pintar el resultado.
3. **No** → lo de antes: `numeroAnterior = Number(pantalla.textContent)`.
4. Lo demás (guardar operador y levantar la bandera) se queda igual, después del `if/else`.

La segunda condición evita que, si pulsáis `+` dos veces seguidas, se haga una cuenta que nadie ha pedido.



Comprobadlo: `5` `+` `3` `+` → se ve `8`.
Seguid con `2` `=` → `10`.
Y `5` `+` `+` `3` `=` → `8`.



¿Atascados? La solución de este paso está en el anexo, apartado **S0-7**. Intentadlo antes de mirar.

☐ **¡Hecho! 5 + 3 + 2 = 10**

SOLUCIÓN 0 · PASO 8 DE 9 · HITO 6



El botón de borrar



Objetivo: al pulsar `C`, la pantalla vuelve a `0` y la calculadora olvida la operación pendiente.

Dónde: bloque `TODO 3`, dentro del `switch (valor)`, debajo del `case 'igual'`.

Pistas, en orden

1. Añadid un `case 'borrar':`.
2. Poned la pantalla a `'0'`.
3. `operadorPendiente = null` y `empezarNumero = true`.
4. `break`.



Comprobadlo: escribid `999`, pulsad `C` → `0`.
Escribid `5` `+` `C` y luego `3` `=` → se ve `3`: la suma pendiente se ha olvidado.



Si no funciona: si al pulsar `C` se hace una cuenta, os falta el `break` del `case 'igual'`... o el de `'borrar'` está encima y se cae al de abajo.



¿Atascados? La solución de este paso está en el anexo, apartado **S0-8**. Intentadlo antes de mirar.

☐ **¡Hecho! 999 C → 0**

SOLUCIÓN 0 · PASO 9 DE 9 · HITO 7



Errores y decimales



Objetivo: dividir entre cero muestra `Error`, después de un error no se sigue operando, y los decimales no salen con quince cifras.

Parte A · No seguir operando tras un error

Dónde: bloque `TODO 2` (operadores), la línea de `operadorPendiente = valor`.

Cambiadla por un ternario[12]: si `numeroAnterior` es `'Error'`, el operador pendiente vuelve a `null`; si no, se guarda `valor`.

Parte B · Redondear (no viene en la plantilla)

Dónde: dentro de `calcular()`, justo antes del `return`.

Si el resultado es un número (`typeof resultado === 'number'`[13]), redondeadlo a 8 decimales: multiplicad por `100000000`, redondead con `Math.round`[14] y volved a dividir.



Comprobadlo: `8 / 0 =` → `Error`.
`8 / 0 +` → `Error`, y el `+` no queda pendiente.
`10 / 3 =` → `3.33333333` (antes salía `3.3333333333333335`).



¿Atascados? La solución de este paso está en el anexo, apartado **S0-9**. Intentadlo antes de mirar.



Ojo: el `typeof` está para no intentar redondear el texto `'Error'`. Sin él, `Math.round` devolvería `NaN`.

☐ **¡Hecho! Error y 3.33333333**



1

SOLUCIÓN 1 · CARPETA VERSION-1-ESCUCHADORES-INDIVIDUALES

Un escuchador por botón

Ahora el número de la pantalla se guarda en una variable, `operandoActual`, y cada tarea va en su propia función. Los botones solo pasan el recado: «me han pulsado, encárgate tú».



Para entenderlo: en un restaurante, el camarero no cocina. Toma nota y se la pasa a la cocina. Los botones son los camareros; las funciones, los cocineros.

Preparativos | 1 · Hito 1 | 2 · Hito 2 | 3 · Hito 3 | 4 · Hito 4 | 5 · Hito 5 | 6 · Hito 6 | 7 · Hito 7

SOLUCIÓN 1 · PREPARATIVOS (YA ESCRITOS)



Lo que ya tenéis en la plantilla

```
const display = document.getElementById('display');
const botonesNumeros    = document.querySelectorAll('[data-numero]');
const botonesOperadores = document.querySelectorAll('[data-operador]');
const botonIgual  = document.querySelector('[data-accion="igual"]');
const botonBorrar = document.querySelector('[data-accion="borrar"]');

let operandoActual = '0';
let operandoAnterior = null;
let operacion = null;
let reiniciarDisplay = false;

function actualizarDisplay() {
    display.textContent = operandoActual;
}
```

`getElementById` busca un elemento por su `id` [15]. El igual y el borrar se buscan con `querySelector`, en singular, porque solo hay uno de cada.

`operandoActual` empieza en `'0'`, **con comillas**. Lo guardamos como texto para poder pegar dígitos: `'1' + '2'` → `'12'`.

`actualizarDisplay()` es la **única** función que toca la pantalla. Regla para toda la solución: cambiáis la variable y luego llamáis a `actualizarDisplay()`.



Comprobadlo: en la consola: `botonesNumeros.length` → `10`, `botonIgual` → os marca el botón `=`.

☐ **¡Hecho! Lo he leído y lo entiendo**

SOLUCIÓN 1 · PASO 1 DE 7 · HITO 1



Escribir números en pantalla



Objetivo: al pulsar los números, aparecen en pantalla. Sin `07`.

Dónde: `TODO 1.1` (función `agregarNumero`) y `TODO 1.2` (el `forEach` de debajo).

Pistas para `agregarNumero(numero)`

1. Si `reiniciarDisplay` es `true`: `operandoActual = numero` y bajad la bandera.
2. Si no: si `operandoActual` es `'0'`, sustituidlo por `numero`; si no, pegad `numero` detrás.
3. Al final, `actualizarDisplay()`.

Pistas para el `TODO 1.2`

1. Dentro del `forEach`, añadid a `boton` un `addEventListener('click', ...)`.
2. Dentro del escuchador, una sola línea: `agregarNumero(boton.dataset.numero)`.



Comprobadlo: `1` `2` `3` → `123`. Recargad: `0` `0` `7` → `7`.



Si no funciona: si la variable cambia pero la pantalla no (comprobadlo escribiendo `operandoActual` en la consola), os habéis olvidado de llamar a `actualizarDisplay()`.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-1**. Intentadlo antes de mirar.

- [ ] **¡Hecho! Escribo 123**

SOLUCIÓN 1 · PASO 2 DE 7 · HITO 2



Guardar el primer número al pulsar un operador



Objetivo: al pulsar `+`, se apunta el número y la operación, y el siguiente dígito empieza un número nuevo.

Dónde: `TODO 2.1` (función `seleccionarOperador`) y `TODO 2.2` (su `forEach`).

Pistas para `seleccionarOperador(nuevoOperador)`

1. Saltaos por ahora el punto «(Opcional por ahora)». Lo haremos en el hito 5.
2. `operandoAnterior = operandoActual`
3. `operacion = nuevoOperador`
4. `reiniciarDisplay = true`

Pistas para el `TODO 2.2`

Igual que en el hito 1, pero llamando a `seleccionarOperador(boton.dataset.operador)`.



Comprobadlo: escribid `12`, pulsad `+`, escribid `5` → se ve `5`.
En la consola: `operandoAnterior` → `'12'` y `operacion` → `'+'`.



Si no funciona: si se ve `125`, la bandera no se está levantando. Si `operandoAnterior` sale `null`, revisad que el escuchador llama a la función correcta.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-2**. Intentadlo antes de mirar.

☐ **¡Hecho! Veo el 5**

SOLUCIÓN 1 · PASO 3 DE 7 · HITO 3



Resolver la suma con el igual



Objetivo: `7 + 8 =` muestra `15`. De momento, solo la suma.

Dónde: `TODO 3.1` (función `calcular`) y `TODO 3.2` (escuchador de `botonIgual`).

Pistas para `calcular()`

1. Primera línea: si `operacion` o `operandoAnterior` son `null`, `return`. Es el portero: si no hay nada que calcular, fuera.
2. Convertid los dos operandos a número con `parseFloat()` [16] y guardadlos en `anterior` y `actual`.
3. `let resultado;` y un `switch (operacion)` con `case '+'` y un `default: return;`.
4. Guardad `resultado.toString()` [17] en `operandoActual`.
5. Dejadlo todo limpio: `operacion = null`, `operandoAnterior = null`, `reiniciarDisplay = true`.
6. `actualizarDisplay()`.

Para el `TODO 3.2`: `botonIgual.addEventListener('click', ...)` que llame a `calcular()`. Aquí no hay `forEach`: es un solo botón.



Comprobadlo: `7` `+` `8` `=` → `15`. Pulsad `=` sin operador: no pasa nada.



Si no funciona: si sale `78`, falta el `parseFloat`. Si la consola dice *Cannot read properties of undefined (reading 'toString')*, os falta el `default: return;` y habéis probado una resta antes de tiempo.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-3**. Intentadlo antes de mirar.

☐ **¡Hecho! 7 + 8 = 15**

SOLUCIÓN 1 · PASO 4 DE 7 · HITO 4



Resta, multiplicación y división



Objetivo: que las cuatro operaciones funcionen.

Dónde: dentro del `switch` de `calcular()`, entre el `case '+'` y el `default`.

Pistas, en orden

1. Copiad el `case '+'` tres veces.
2. Cambiad el símbolo del `case` y el de la cuenta: `'-'`, `'*'` y `'/'`.
3. No os olvidéis del `break` en cada uno.

La división entre cero la arreglaremos en el hito 7. Si la probáis ahora, veréis `Infinity`: JavaScript no da error, devuelve «infinito».



Comprobadlo: `9 - 4 =` → `5`
`6 * 3 =` → `18`
`20 / 4 =` → `5`



Si no funciona: si `9 - 4 =` da `13`, habéis copiado el `case` pero no habéis cambiado el `+` de la cuenta.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-4**. Intentadlo antes de mirar.

☐ **¡Hecho! Salen 5, 18 y 5**

SOLUCIÓN 1 · PASO 5 DE 7 · HITO 5



Operaciones encadenadas



Objetivo: `5 + 3 +` muestra ya el subtotal `8`.

Dónde: el punto «(Opcional por ahora)» del `TODO 2.1`, al principio de `seleccionarOperador`.

Pistas, en orden

1. Antes de las tres líneas que ya tenéis, preguntad: ¿hay una operación pendiente (`operacion ≠ null`) **y** el usuario ya ha escrito el segundo número (`!reiniciarDisplay`)?
2. Si es así, llamad a `calcular()`. Nada más.

¿Y por qué funciona? Porque `calcular()` deja el subtotal en `operandoActual`, y la línea siguiente lo guarda como `operandoAnterior`. Las piezas encajan solas.

El `!` significa «no»: `!reiniciarDisplay` se lee «la bandera no está levantada».



Comprobadlo: `5` `+` `3` `+` → `8`. Luego `2` `=` → `10`.
`2` `*` `3` `-` `1` `=` → `5`.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-5**. Intentadlo antes de mirar.

☐ **¡Hecho! 5 + 3 + 2 = 10**



El botón de borrar



Objetivo: `C` deja la calculadora como recién encendida.

Dónde: `TODO 6.1` (función `borrar`) y `TODO 6.2` (escuchador de `botonBorrar`).

Pistas, en orden

1. Poned las cuatro variables como están al principio del archivo: `'0'`, `null`, `null` y `false`.
2. `actualizarDisplay()`.
3. Conectad `botonBorrar` con un `addEventListener` que llame a `borrar()`.

Truco: subid al principio del archivo y copiad los valores iniciales. Así no os equivocáis.



Comprobadlo: `999` `C` → `0`.
`5` `+` `C` `3` `=` → `3`.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-6**. Intentadlo antes de mirar.

☐ **¡Hecho! 999 C → 0**

SOLUCIÓN 1 · PASO 7 DE 7 · HITO 7



Errores y decimales



Objetivo: dividir entre cero muestra `Error` y los resultados no salen con quince decimales.

Parte A · División entre cero

Dónde: el `case '/'` de `calcular()`.

Si `actual === 0`, el resultado es `'Error'`; si no, dividid. Podéis usar un ternario o un `if/else`.

Parte B · Redondeo (no viene en la plantilla)

Dónde: en `calcular()`, después del `switch` y antes de `operandoActual = resultado.toString()`.

Si `typeof resultado === 'number'` [13], redondeadlo: `Math.round(resultado * 100000000) / 100000000` [14].



Comprobadlo: `8 / 0 =` → `Error`.
`10 / 3 =` → `3.33333333`.



Ojo: después de un `Error`, pulsad `C` antes de seguir. Si pulsáis `+ 3 =` directamente, veréis `NaN` («no es un número»). Arreglarlo es un buen reto extra.



¿Atascados? La solución de este paso está en el anexo, apartado **S1-7**. Intentadlo antes de mirar.

☐ **¡Hecho! Error y 3.33333333**



2

SOLUCIÓN 2 · CARPETA VERSION-2-DELEGACION-EVENTOS

Delegación de eventos

Las funciones son las mismas que en la Solución 1. La diferencia es que no hay 16 escuchadores: hay **uno solo**, en el contenedor `.teclado`. Por eso aquí empezamos por el escuchador.

1 · El conserje | 2 · Hito 1 | 3 · Hito 2 | 4 · Hito 3 | 5 · Hito 4 | 6 · Hito 5 | 7 · Hito 6 | 8 · Hito 7

SOLUCIÓN 2 · LA IDEA ANTES DEL CÓDIGO



El clic sube como una burbuja

Cuando hacéis clic en un botón, el aviso no se queda en el botón: sube a su padre (`.teclado`), luego al padre de ese, y así hasta arriba. Se llama **burbujeo**[18].

Si ponemos un escuchador en `.teclado`, se enterará de los clics de **todos** sus botones.



Para entenderlo: el conserje de un edificio. Todos los paquetes llegan a la portería; el conserje mira la etiqueta y los sube al piso que toca. No hace falta un vigilante en cada puerta.

Dos cosas que el conserje tiene que averiguar

1. **¿Qué botón han pulsado?** `evento.target` es el elemento exacto donde cayó el clic[19]. `.closest('button')` sube desde ahí hasta encontrar el botón[20]. Si el clic cayó en un hueco entre teclas, devuelve `null`.
2. **¿Qué etiqueta lleva?** Lo leemos de `boton.dataset`. Las etiquetas que no tiene valen `undefined`.

SOLUCIÓN 2 · PASO 1 DE 8 · PREPARATIVOS



Que el conserje os diga qué ha visto



Objetivo: ver en la consola qué botón se ha pulsado, antes de programar nada más.

Dónde: el bloque final de la plantilla, `teclado.addEventListener( … )`, debajo de `// Escribe las condiciones aquí:`.

La plantilla ya trae estas líneas:

```
const boton = evento.target.closest('button');
if (!boton) return;

const { numero, operador, accion } = boton.dataset;
```

La última es una **desestructuración**[21]: saca tres propiedades de `dataset` de golpe. Equivale a escribir `const numero = boton.dataset.numero;` y lo mismo con las otras dos.

Vuestra tarea

Añadid una línea provisional: `console.log(numero, operador, accion);` [22]



Comprobadlo: con la consola abierta, pulsad:

- `7` → `7 undefined undefined`
- `+` → `undefined + undefined`
- `=` → `undefined undefined igual`
- `C` → `undefined undefined borrar`



Si no funciona: si no sale nada, comprobad que la consola no tiene filtros y que `teclado` no es `null` (escribid `teclado` en la consola).

Fijaos: **un único escuchador** ya «oye» los 16 botones. Eso es la delegación.

- [ ] **¡Hecho! La consola me lo cuenta todo**

SOLUCIÓN 2 · PASO 2 DE 8 · HITO 1



Escribir números en pantalla



Objetivo: al pulsar los números, aparecen en pantalla. Sin `07`.

Dónde: `TODO 1.1` (función `agregarNumero`) y una condición en el escuchador.

Pistas para `agregarNumero(numero)`

Es idéntica a la de la Solución 1: si la bandera está levantada, sustituid; si hay un `'0'`, sustituid; si no, pegad. Y `actualizarDisplay()` al final.

Pistas para el escuchador

1. Debajo del `console.log`, escribid: si `numero !== undefined`, llamad a `agregarNumero(numero)`.

¿Por qué `!== undefined` y no `if (numero)`? Porque dice exactamente lo que queremos preguntar: «¿este botón tiene `data-numero`?». Con un `if (numero)` a secas, un valor vacío o un `0` numérico se tratarían como falsos[23].



Comprobadlo: `1` `2` `3` → `123`. Recargad: `0` `0` `7` → `7`.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-2**. Intentadlo antes de mirar.

☐ **¡Hecho! Escribo 123**

SOLUCIÓN 2 · PASO 3 DE 8 · HITO 2



Guardar el primer número al pulsar un operador



Objetivo: `12 + 5` muestra `5` y el `12` queda apuntado.

Dónde: `TODO 2.1` (función `seleccionarOperador`) y una rama nueva en el escuchador.

Pistas, en orden

1. En `seleccionarOperador`, saltaos de momento el punto «(Opcional por ahora)».
2. `operandoAnterior = operandoActual`, `operacion = nuevoOperador`, `reiniciarDisplay = true`.
3. En el escuchador, convertid el `if` del paso anterior en `if … else if`: si `operador !== undefined`, llamad a `seleccionarOperador(operador)`.



Comprobadlo: `12` `+` `5` → `5`.
En la consola: `operandoAnterior` → `'12'`, `operacion` → `'+'`.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-3**. Intentadlo antes de mirar.

☐ **¡Hecho! Veo el 5**

SOLUCIÓN 2 · PASO 4 DE 8 · HITO 3



Resolver la suma con el igual



Objetivo: `7 + 8 =` muestra `15`.

Dónde: `TODO 3.1` (función `calcular`) y otra rama en el escuchador.

Pistas para `calcular()`

1. El portero: si `operacion` o `operandoAnterior` son `null`, `return`.
2. `parseFloat()` [16] a los dos operandos.
3. `switch` con `case '+'` y `default: return;`.
4. `operandoActual = resultado.toString()` [17], limpiar `operacion` y `operandoAnterior`, levantar la bandera y `actualizarDisplay()`.

Pista para el escuchador

`else if (accion === 'igual')` → `calcular()`.



Comprobadlo: `7` `+` `8` `=` → `15`.



Si no funciona: si sale `78`, falta el `parseFloat`. Si la consola dice *Cannot read properties of undefined (reading 'toString')*, falta el `default: return;`.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-4**. Intentadlo antes de mirar.

☐ **¡Hecho! 7 + 8 = 15**

SOLUCIÓN 2 · PASO 5 DE 8 · HITO 4



Resta, multiplicación y división



Objetivo: que las cuatro operaciones funcionen.

Dónde: el `switch` de `calcular()`.

Igual que en la Solución 1: copiad el `case '+'` tres veces y cambiad los símbolos. El escuchador **no se toca**: todos los operadores ya pasan por `seleccionarOperador`.

La división entre cero dará `Infinity` hasta el hito 7.



Comprobadlo: `9 - 4 =` → `5` · `6 * 3 =` → `18` · `20 / 4 =` → `5`



¿Atascados? La solución de este paso está en el anexo, apartado **S2-5**. Intentadlo antes de mirar.

- [ ] **¡Hecho! Salen 5, 18 y 5**

SOLUCIÓN 2 · PASO 6 DE 8 · HITO 5



Operaciones encadenadas



Objetivo: `5 + 3 +` muestra ya el subtotal `8`.

Dónde: el punto «(Opcional por ahora)» del `TODO 2.1`, al principio de `seleccionarOperador`.

Exactamente igual que en la Solución 1: si hay operación pendiente **y** la bandera está bajada, llamad a `calcular()` antes de lo demás.



Comprobadlo: `5` `+` `3` `+` → `8`. Luego `2` `=` → `10`.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-6**. Intentadlo antes de mirar.

- [ ] **¡Hecho! 5 + 3 + 2 = 10**

SOLUCIÓN 2 · PASO 7 DE 8 · HITO 6



El botón de borrar



Objetivo: `C` deja la calculadora como recién encendida.

Dónde: `TODO 6.1` (función `borrar`) y la última rama del escuchador.

1. En `borrar()`: `'0'`, `null`, `null`, `false` y `actualizarDisplay()`.
2. En el escuchador: `else if (accion === 'borrar')` → `borrar()`.
3. Ya tenéis las cuatro ramas: **quitad el** `console.log` provisional del paso 1.



Comprobadlo: `999` `C` → `0`.



¿Atascados? La solución de este paso está en el anexo, apartado **S2-7**. Intentadlo antes de mirar.

☐ **¡Hecho! 999 C → 0**



Errores, decimales... y el superpoder de la delegación



Objetivo: `8 / 0 =` muestra `Error` y `10 / 3 =` muestra `3.33333333`.

Dónde: el `case '/'` de `calcular()` y, después del `switch`, el redondeo.

Es el mismo código que en el hito 7 de la Solución 1: ternario para el `'Error'` y `Math.round` [14] si `typeof resultado === 'number'` [13].



¿Atascados? La solución de este paso está en el anexo, apartado **S2-8**. Intentadlo antes de mirar.

☐ **¡Hecho! Error y 3.33333333**

Prueba extra: botones nuevos sin tocar el JS

Con la calculadora cargada, pegad esto en la consola. Añade una tecla `00` al teclado:

```
teclado.insertAdjacentHTML('beforeend',
  '<button class="tecla" data-numero="00">00</button>');
```

Pulsadla: funciona. El conserje no necesita que nadie le avise de que hay un piso nuevo.

Haced lo mismo en la Solución 1 (cambiando `teclado` por `document.querySelector('.teclado')`). La tecla aparece, pero no hace nada: nadie le ha puesto un escuchador.

COMPROBACIÓN FINAL · PARA LAS TRES SOLUCIONES



Antes de entregar, pasad esta lista

HITO	TECLAS	DEBE SALIR	HECHO
1	`1 2 3`	`123`	☐
1	`0 0 7`	`7`	☐
2	`15 + 4`	`4`	☐
3	`7 + 8 =`	`15`	☐
4	`9 − 4 =` · `6 * 3 =` · `20 / 4 =`	`5` · `18` · `5`	☐
5	`5 + 3 +` y luego `2 =`	`8` y luego `10`	☐
6	`999 C`	`0`	☐
7	`8 / 0 =`	`Error`	☐
7	`10 / 3 =`	`3.33333333`	☐
—	Consola abierta durante todas las pruebas	Ningún error en rojo	☐

Y quitad los `console.log` provisionales. Funcionan, sí, pero ensucian la entrega.

SI ALGO FALLA · TRADUCCIÓN DE LA CONSOLA



Lo que os dice la consola y lo que significa

MENSAJE EN ROJO	QUÉ SIGNIFICA	DÓNDE MIRAR
`Uncaught ReferenceError: agregarNumro is not defined`	Usáis un nombre que no existe. Casi siempre es una errata.	Copiad el nombre desde donde se declara, no lo escribáis de memoria.
`Uncaught SyntaxError: Unexpected token '}'`	Sobra o falta una llave, un paréntesis o una coma.	La línea que indica la consola o la de justo encima. El editor os marca las parejas de llaves.
`Cannot read properties of null (reading 'addEventListener')`	El selector no ha encontrado nada y la variable vale `null`.	El selector: ¿`data-accion="igual"` bien escrito? ¿Comillas dentro de comillas?
`Cannot read properties of undefined (reading 'toString')`	`resultado` no tiene valor: ningún `case` ha coincidido.	El `switch` de `calcular()` y su `default: return;`.
`Assignment to constant variable`	Intentáis cambiar una variable declarada con `const`.	Las variables de la memoria van con `let`.
Sin error, pero no cambia nada	No habéis guardado o no habéis recargado. Lo de siempre.	`Ctrl`+`S` y `F5`.

Un consejo que vale para toda vuestra vida programando: leed el mensaje de error entero. Os dice qué ha pasado y en qué línea. Ignorarlo es como ir al médico y taparse los oídos.



Solución 0 · La de clase

Solo para comprobar, no para copiar. Si miráis aquí antes de intentarlo, os estáis haciendo trampas al solitario.

S0-1 · Solución 0 · Paso 1 de 9 · Preparativos — La función `calcular()`

```
function calcular(numero1, numero2, operacion) {
    let resultado = 0;

    switch (operacion) {
        case '+':
            resultado = numero1 + numero2;
            break;
        case '-':
            resultado = numero1 - numero2;
            break;
        case '*':
            resultado = numero1 * numero2;
            break;
        case '/':
            if (numero2 === 0) {
                resultado = 'Error';
            } else {
                resultado = numero1 / numero2;
            }
            break;
    }

    return resultado;
}
```

S0-3 · Solución 0 · Paso 3 de 9 · Hito 1 — Escribir números en pantalla

```
numeros.forEach(numero => {
    numero.addEventListener('click', () => {
        const valor = numero.dataset.numero;

        if (pantalla.textContent === '0' || empezarNumero) {
            pantalla.textContent = valor;
            empezarNumero = false;
        } else {
            pantalla.textContent += valor;
        }
    });
});
```

S0-4 · Solución 0 · Paso 4 de 9 · Hito 2 — Guardar el primer número al pulsar un operador

```
operadores.forEach(operador => {
    operador.addEventListener('click', () => {
        const valor = operador.dataset.operador;

        numeroAnterior = Number(pantalla.textContent);
        operadorPendiente = valor;
        empezarNumero = true;
    });
});
```

S0-5 · Solución 0 · Paso 5 de 9 · Hito 3 — Resolver la cuenta con el igual

```
acciones.forEach(accion => {
    accion.addEventListener('click', () => {
        const valor = accion.dataset.accion;

        switch (valor) {
            case 'igual':
                if (operadorPendiente !== null) {
                    numeroActual = Number(pantalla.textContent);
                    numeroAnterior = calcular(numeroAnterior, numeroActual, operadorPendiente);
                    pantalla.textContent = numeroAnterior;
                }
                operadorPendiente = null;
                empezarNumero = true;
                break;
        }
    });
});
```

S0-7 · Solución 0 · Paso 7 de 9 · Hito 5 — Operaciones encadenadas

```
operadores.forEach(operador => {
    operador.addEventListener('click', () => {
        const valor = operador.dataset.operador;

        if (operadorPendiente !== null && empezarNumero !== true) {
            numeroActual = Number(pantalla.textContent);
            numeroAnterior = calcular(numeroAnterior, numeroActual, operadorPendiente);
            pantalla.textContent = numeroAnterior;
        } else {
            numeroAnterior = Number(pantalla.textContent);
        }

        operadorPendiente = valor;
        empezarNumero = true;
    });
});
```

S0-8 · Solución 0 · Paso 8 de 9 · Hito 6 — El botón de borrar

```
            case 'borrar':
                pantalla.textContent = '0';
                operadorPendiente = null;
                empezarNumero = true;
                break;
```

S0-9 · Solución 0 · Paso 9 de 9 · Hito 7 — Errores y decimales

```
// Parte A: en los operadores
operadorPendiente = (numeroAnterior === 'Error') ? null : valor;

// Parte B: en calcular(), antes del return
if (typeof resultado === 'number') {
    resultado = Math.round(resultado * 100000000) / 100000000;
}
return resultado;
```

ANEXO · SOLUCIONES



Solución 1 · Un escuchador por botón

Solo para comprobar, no para copiar. Si miráis aquí antes de intentarlo, os estáis haciendo trampas al solitario.

S1-1 · Solución 1 · Paso 1 de 7 · Hito 1 — Escribir números en pantalla

```
function agregarNumero(numero) {
    if (reiniciarDisplay) {
        operandoActual = numero;
        reiniciarDisplay = false;
    } else {
        operandoActual = (operandoActual === '0')
            ? numero
            : operandoActual + numero;
    }
    actualizarDisplay();
}

botonesNumeros.forEach((boton) => {
    boton.addEventListener('click', () => {
        agregarNumero(boton.dataset.numero);
    });
});
```

El `? :` es un ternario[12], un `if/else` en una línea. Si os resulta más claro con `if/else`, escribidlo así: funciona igual.

S1-2 · Solución 1 · Paso 2 de 7 · Hito 2 — Guardar el primer número al pulsar un operador

```
function seleccionarOperador(nuevoOperador) {
    operandoAnterior = operandoActual;
    operacion = nuevoOperador;
    reiniciarDisplay = true;
}

botonesOperadores.forEach((boton) => {
    boton.addEventListener('click', () => {
        seleccionarOperador(boton.dataset.operador);
    });
});
```

S1-3 · Solución 1 · Paso 3 de 7 · Hito 3 — Resolver la suma con el igual

```
function calcular() {
    if (operacion === null || operandoAnterior === null) return;

    const anterior = parseFloat(operandoAnterior);
    const actual = parseFloat(operandoActual);
    let resultado;

    switch (operacion) {
        case '+':
            resultado = anterior + actual;
            break;
        default:
            return;
    }

    operandoActual = resultado.toString();
    operacion = null;
    operandoAnterior = null;
    reiniciarDisplay = true;
    actualizarDisplay();
}

botonIgual.addEventListener('click', () => {
    calcular();
});
```

S1-4 · Solución 1 · Paso 4 de 7 · Hito 4 — Resta, multiplicación y división

```
    switch (operacion) {
        case '+':
            resultado = anterior + actual;
            break;
        case '-':
            resultado = anterior - actual;
            break;
        case '*':
            resultado = anterior * actual;
            break;
        case '/':
            resultado = anterior / actual;
            break;
        default:
            return;
    }
```

S1-5 · Solución 1 · Paso 5 de 7 · Hito 5 — Operaciones encadenadas

```
function seleccionarOperador(nuevoOperador) {
    if (operacion !== null && !reiniciarDisplay) {
        calcular();
    }
    operandoAnterior = operandoActual;
    operacion = nuevoOperador;
    reiniciarDisplay = true;
}
```

S1-6 · Solución 1 · Paso 6 de 7 · Hito 6 — El botón de borrar

```
function borrar() {
    operandoActual = '0';
    operandoAnterior = null;
    operacion = null;
    reiniciarDisplay = false;
    actualizarDisplay();
}

botonBorrar.addEventListener('click', () => {
    borrar();
});
```

S1-7 · Solución 1 · Paso 7 de 7 · Hito 7 — Errores y decimales

```
        case '/':
            resultado = (actual === 0) ? 'Error' : anterior / actual;
            break;
        default:
            return;
    }

    if (typeof resultado === 'number') {
        resultado = Math.round(resultado * 100000000) / 100000000;
    }

    operandoActual = resultado.toString();
```



Solución 2 · Delegación de eventos

Solo para comprobar, no para copiar. Si miráis aquí antes de intentarlo, os estáis haciendo trampas al solitario.

S2-2 · Solución 2 · Paso 2 de 8 · Hito 1 — Escribir números en pantalla

```
function agregarNumero(numero) {
    if (reiniciarDisplay) {
        operandoActual = numero;
        reiniciarDisplay = false;
    } else {
        operandoActual = (operandoActual === '0')
            ? numero
            : operandoActual + numero;
    }
    actualizarDisplay();
}

// Dentro del escuchador del teclado:
if (numero !== undefined) {
    agregarNumero(numero);
}
```

S2-3 · Solución 2 · Paso 3 de 8 · Hito 2 — Guardar el primer número al pulsar un operador

```
function seleccionarOperador(nuevoOperador) {
    operandoAnterior = operandoActual;
    operacion = nuevoOperador;
    reiniciarDisplay = true;
}

// Dentro del escuchador del teclado:
if (numero !== undefined) {
    agregarNumero(numero);
} else if (operador !== undefined) {
    seleccionarOperador(operador);
}
```

S2-4 · Solución 2 · Paso 4 de 8 · Hito 3 — Resolver la suma con el igual

```
function calcular() {
    if (operacion === null || operandoAnterior === null) return;

    const anterior = parseFloat(operandoAnterior);
    const actual = parseFloat(operandoActual);
    let resultado;

    switch (operacion) {
        case '+':
            resultado = anterior + actual;
            break;
        default:
            return;
    }

    operandoActual = resultado.toString();
    operacion = null;
    operandoAnterior = null;
    reiniciarDisplay = true;
    actualizarDisplay();
}

// En el escuchador, una rama más:
if (numero !== undefined) {
    agregarNumero(numero);
} else if (operador !== undefined) {
    seleccionarOperador(operador);
} else if (accion === 'igual') {
    calcular();
}
```

S2-5 · Solución 2 · Paso 5 de 8 · Hito 4 — Resta, multiplicación y división

```
    switch (operacion) {
        case '+':
            resultado = anterior + actual;
            break;
        case '-':
            resultado = anterior - actual;
            break;
        case '*':
            resultado = anterior * actual;
            break;
        case '/':
            resultado = anterior / actual;
            break;
        default:
            return;
    }
```

S2-6 · Solución 2 · Paso 6 de 8 · Hito 5 — Operaciones encadenadas

```
function seleccionarOperador(nuevoOperador) {
    if (operacion !== null && !reiniciarDisplay) {
        calcular();
    }
    operandoAnterior = operandoActual;
    operacion = nuevoOperador;
    reiniciarDisplay = true;
}
```

S2-7 · Solución 2 · Paso 7 de 8 · Hito 6 — El botón de borrar

```
function borrar() {
    operandoActual = '0';
    operandoAnterior = null;
    operacion = null;
    reiniciarDisplay = false;
    actualizarDisplay();
}

teclado.addEventListener('click', (evento) => {
    const boton = evento.target.closest('button');
    if (!boton) return;

    const { numero, operador, accion } = boton.dataset;

    if (numero !== undefined) {
        agregarNumero(numero);
    } else if (operador !== undefined) {
        seleccionarOperador(operador);
    } else if (accion === 'igual') {
        calcular();
    } else if (accion === 'borrar') {
        borrar();
    }
});
```

S2-8 · Solución 2 · Paso 8 de 8 · Hito 7 — Errores, decimales... y el superpoder de la delegación

```
        case '/':
            resultado = (actual === 0) ? 'Error' : anterior / actual;
            break;
        default:
            return;
    }

    if (typeof resultado === 'number') {
        resultado = Math.round(resultado * 100000000) / 100000000;
    }
```



Fuentes y Documentación Oficial

1. MDN Web Docs — ¿Cuáles son las herramientas de desarrollo del navegador?
2. MDN Web Docs — HTMLElement: dataset property
3. MDN Web Docs — Document.querySelectorAll()
4. MDN Web Docs — Document.querySelector()
5. MDN Web Docs — NodeList.prototype.forEach()
6. MDN Web Docs — element.addEventListener
7. MDN Web Docs — Node.textContent
8. MDN Web Docs — switch
9. MDN Web Docs — if...else
10. MDN Web Docs — Operador lógico OR (||)
11. MDN Web Docs — Number
12. MDN Web Docs — Operador condicional (ternario)
13. MDN Web Docs — typeof
14. MDN Web Docs — Math.round()
15. MDN Web Docs — document.getElementById
16. MDN Web Docs — parseFloat()
17. MDN Web Docs — Number.prototype.toString()
18. MDN Web Docs — Burbujeo de eventos
19. MDN Web Docs — Event.target
20. MDN Web Docs — Element.closest()
21. MDN Web Docs — La desestructuración
22. MDN Web Docs — Console.log()
23. MDN Web Docs — Truthy